NestedIOFunction#

https://pytorch.org/docs/stable/generated/torch.autograd.function.NestedIOFu

Description:

This class is here only for backward compatibility reasons. Use Function instead of this for any new use case. Shared backward utility. User defined backward. Shared forward utility.

Function Signature

```
class torch.autograd.function.NestedIOFunction(*args,
**kwargs)[source]#
backward(*gradients)[source]#
Return type
backward_extended(*grad_output)[source]#
forward(*args)[source]#
```

Returns:

Any

Code Examples

```
>>> class Func(torch.autograd.Function):
>>>     @staticmethod
>>>     def forward(ctx, x: torch.Tensor, y: torch.Tensor, z:
int):
>>>         ctx.save_for_backward(x, y)
>>>         ctx.save_for_forward(x, y)
>>>         ctx.z = z
>>>         return x * y * z
>>>
>>>     @staticmethod
>>>     def jvp(ctx, x_t, y_t, _):
>>>         x, y = ctx.saved_tensors
>>>         z = ctx.z
>>>         return z * (y * x_t + x * y_t)
>>>
>>>     @staticmethod
>>>     def vjp(ctx, grad_out):
>>>         x, y = ctx.saved_tensors
>>>         z = ctx.z
>>>         return z * grad_out * y, z * grad_out * x, None
>>>
>>>     a = torch.tensor(1., requires_grad=True,
dtype=torch.double)
>>>     t = torch.tensor(1., dtype=torch.double)
>>>     b = torch.tensor(2., requires_grad=True,
dtype=torch.double)
>>>     c = 4
>>>
>>>     with fwAD.dual_level():
>>>         a_dual = fwAD.make_dual(a, t)
>>>         d = Func.apply(a_dual, b, c)
```

```
>>> class SimpleFunc(Function):
>>>     @staticmethod
>>>     def forward(ctx, x):
>>>         return x.clone(), x.clone()
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, g1, g2):
>>>         return g1 + g2  # No check for None necessary
>>>
>>> # We modify SimpleFunc to handle non-materialized grad
outputs
>>> class Func(Function):
>>>     @staticmethod
>>>     def forward(ctx, x):
>>>         ctx.set_materialize_grads(False)
>>>         ctx.save_for_backward(x)
>>>         return x.clone(), x.clone()
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, g1, g2):
>>>         x, = ctx.saved_tensors
>>>         grad_input = torch.zeros_like(x)
>>>         if g1 is not None:  # We must check for None now
>>>             grad_input += g1
>>>         if g2 is not None:
>>>             grad_input += g2
>>>         return grad_input
>>>
>>> a = torch.tensor(1., requires_grad=True)
>>> b, _ = Func.apply(a)  # induces g2 to be undefined
```

Detailed Documentation

Documentation

This class is here only for backward compatibility reasons. Use Function instead of this for any new use case.

Shared backward utility.

User defined backward.

Shared forward utility.

User defined forward.

Define a formula for differentiating the operation with forward mode automatic differentiation.

This function is to be overridden by all subclasses. It must accept a context ctx as the first argument, followed by as many inputs as the forward() got (None will be passed in for non tensor inputs of the forward function), and it should return as many tensors as there were outputs to forward(). Each argument is the gradient w.r.t the given input, and each returned value should be the gradient w.r.t. the corresponding output. If an output is not a Tensor or the function is not differentiable with respect to that output, you can just pass None as a gradient for that input.

You can use the ctx object to pass any value from the forward to this functions.

See Function.mark_dirty().

See Function.mark_non_differentiable().

See Function.save_for_backward().

Save given tensors for a future call to jvp().

save_for_forward should be called at most once, in either the setup_context() or forward() methods, and all arguments should be tensors.

In jvp(), saved objects can be accessed through the saved_tensors attribute.

Arguments can also be None. This is a no-op.

See Extending torch.autograd for more details on how to use this method.

See Function.saved_tensors().

Set whether to materialize grad tensors. Default is True.

This should be called only from either the setup_context() or forward() methods.

If True, undefined grad tensors will be expanded to tensors full of zeros prior to calling the backward() and jvp() methods.

There are two ways to define the forward pass of an autograd.Function.

Override forward with the signature forward(ctx, *args, **kwargs). setup_context is not overridden. Setting up the ctx for backward happens inside the forward.

Override forward with the signature forward(*args, **kwargs) and override setup_context. Setting up the ctx for backward happens inside setup_context (as opposed to inside the forward)

See torch.autograd.Function.forward() and Extending torch.autograd for more details.

Define a formula for differentiating the operation with backward mode automatic differentiation.

This function is to be overridden by all subclasses. (Defining this function is equivalent to defining the vjp function.)

It must accept a context ctx as the first argument, followed by as many outputs as the forward() returned (None will be passed in for non tensor outputs of the forward function), and it should return as many tensors, as there were inputs to forward(). Each argument is the gradient w.r.t the given output, and each returned value should be the gradient w.r.t. the corresponding input. If an input is not a Tensor or is a Tensor not requiring grads, you can just pass None as a gradient for that input.

The context can be used to retrieve tensors saved during the forward pass. It also has an attribute ctx.needs_input_grad as a tuple of booleans representing whether each input needs gradient. E.g., backward() will have ctx.needs_input_grad[0] = True if the first input to forward() needs gradient computed w.r.t. the output.

Define the behavior for this autograd.Function underneath torch.vmap().

For a torch.autograd.Function() to support torch.vmap(), you must either override this static method, or set generate_vmap_rule to True (you may not do both).

If you choose to override this staticmethod: it must accept

an info object as the first argument. info.batch_size specifies the size of the dimension being vmapped over, while info.randomness is the randomness option passed to torch.vmap().

an in_dims tuple as the second argument. For each arg in args, in_dims has a corresponding Optional[int]. It is None if the arg is not a Tensor or if the arg is not being vmapped over, otherwise, it is an integer specifying what dimension of the Tensor is being vmapped over.

*args, which is the same as the args to forward().

The return of the vmap staticmethod is a tuple of (output, out_dims). Similar to in_dims, out_dims should be of the same structure as output and contain one out_dim per output that specifies if the output has the vmapped dimension and what index it is in.

Please see Extending torch.func with autograd.Function for more details.
